

Multithreading

Conteúdo

- Threads
- Mudança de Estado
- Problema da Seção Crítica
- Mutexes e Variáveis Condicionais
- Deadlocks

Threads

```
type Thread
func Spawn(func ()) Thread
func Join(&Thread)
func Detach(&Thread)
```

```
typedef struct { ... } thrd_t;
int thrd_create(thrd_t*, int(*)(void*), void*);
int thrd_join(thrd_t, int*);
int thrd_detach(thrd_t);
```

```
class Thread {
    public Thread(Runnable);
    public void start();
    public void join();
    public void setDaemon(boolean);
}
```

```
interface Runnable {
    public void run();
}
```

```
class Thread:
    def __init__(self, target: Callable, ...): ...
    def start(self) -> None: ...
    def join(self) -> None: ...
    daemon: bool
```

Threads

Foreground Threads (joined)

- Recursos são liberados completamente apenas após `Join()`;
- Comum em implementações sobre threads de sistemas operacionais.

Background Threads (daemon)

- Recursos são liberados complementamente no término;
- Comum em implementações de threads virtuais;

Threads

```
#include <threads.h>

extern int some_func(void*);

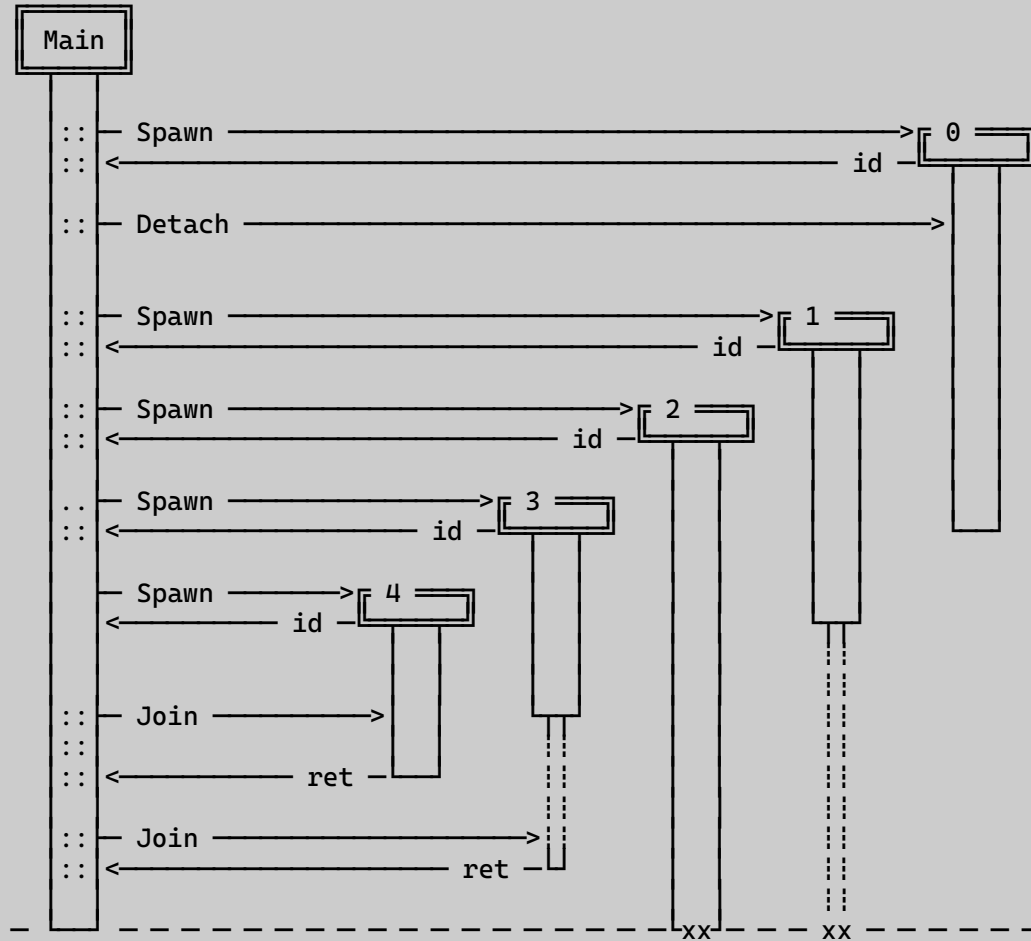
int main() {
    thrd_t threads[5];
    void *envs[5] = { ... };

    thrd_create(&threads[0], some_func, envs[0]);
    thrd_detach(threads[0]);

    thrd_create(&threads[1], some_func, envs[1]);
    thrd_create(&threads[2], some_func, envs[2]);
    thrd_create(&threads[3], some_func, envs[3]);
    thrd_create(&threads[4], some_func, envs[4]);

    thrd_join(threads[4], NULL);
    thrd_join(threads[3], NULL);
}
```

Threads



Threads

Exemplo: Servidor Web

```
func main() {  
    ln Listener  
    if net.Listen(&ln, "0.0.0.0:8080") != nil {  
        fmt.Println("erro: Listen")  
        return  
    }  
  
    for {  
        conn := new(net.Conn)  
        if ln.Accept(conn) != nil {  
            fmt.Println("erro: Accept")  
            continue  
        }  
  
        Spawn(func () { handle(conn) }) <-| Vazamento! |  
    }  
}
```

```
func handle(conn net.Conn) {  
    conn.Write(  
        "HTTP/1.1 200 OK\r\n" +  
        "Content-Length: 12\r\n" +  
        "Content-Type: text/plain\r\n" +  
        "\r\n" +  
        "Hello, World"  
    )  
  
    conn.Close()  
    free(conn)  
}
```

Mudança de Estado

```
func main() {
    N:    const = 2
    Inc:  const = 1_000_000

    counter := 0
    threads []Threads

    for range N {
        t := Spawn(func () {
            increment(&counter, Inc)
        })

        append(&threads, t)
    }

    for i := range N {
        Join(&threads[i])
    }

    fmt.Println("Want:", N * Inc)
    fmt.Println("Got: ", counter)
}
```

```
func increment(counter &int, inc int) {
    for range inc {
        counter = counter + 1
    }
}
```

Mudança de Estado

```
$ gcc main.c -o main
```

```
$ ./main
```

```
Want: 1000000
```

```
Got: 535809
```

```
$ ./main
```

```
Want: 1000000
```

```
Got: 719338
```

```
$ ./main
```

```
Want: 1000000
```

```
Got: 568014
```

Este comportamento é atualmente conhecido como uma condição de corrida.

Mudança de Estado

```
counter = counter + 1
```



```
mov eax, [counter]  
add eax, 1  
mov [counter], eax
```

Mudança de Estado

Thread 1	eax	Thread 2	eax	Memória
> mov eax, [counter]	0			← 0
> add eax, 1	1			0
> mov [counter], eax	1			→ 1
>		mov eax, [counter]	1	← 1
>		add eax, 1	2	1
>		mov [counter], eax	2	→ 2

Mudança de Estado

Thread 1	eax	Thread 2	eax	Memória
> mov eax, [counter]	0			← 0
> add eax, 1	1			0
>		mov eax, [counter]	0	← 0
> mov [counter], eax	1			→ 1
>		add eax, 1	1	1
>		mov [counter], eax	1	→ 1

Mudança de Estado

```
func main() {  
    request queue:Queue[string] := /* ... */  
    result  queue:Queue[string] := /* ... */  
  
    W const int := /* ... */  
    workers []Thread  
  
    for range W {  
        t := Spawn(func () {  
            url      := queue:Dequeue(&request)  
            response := net:Fetch(url)  
            body     := io:ReadAll(response.Body)  
            queue:Enqueue(&result, body)  
        })  
  
        append(&workers, t)  
    }  
  
    for i := range W {  
        Join(&workers[i])  
    }  
  
    /* save work */  
}
```

Problema da Seção Crítica

Seção Crítica é uma área de um programa que, em todo instante, no máximo uma thread pode estar executando.

```
func increment(counter &int, inc int) {  
    for range inc {  
        -----  
        counter = counter + 1  
        -----  
    }  
}
```

} Seção Crítica

Problema da Seção Crítica

(i) **Exclusão Mútua (Mutual Exclusion)** No máximo uma thread é permitida na seção crítica por vez.

(ii) **Livre de Interpassagem (Livelock Freedom)** Se uma ou mais threads desejam adentrar a seção crítica, uma deve eventualmente conseguir.

Problema da Seção Crítica

Solução de Dijkstra, 1965

```
N: const int = /* número de threads */  
b: []bool = [N]bool{false, ..., false}  
c: []bool = [N]bool{false, ..., false}  
k: int = /* 0 <= k < N */
```

```
func Unlock(i int) {  
    c[i] = false  
    b[i] = false  
}
```

```
func Lock(i int) {  
    b[i] = true
```

```
    Loop:
```

```
        for k != i {  
            c[i] = false  
            if !b[k] { k = i }  
        }
```

```
        c[i] = true  
        for j := range N {  
            if j != i && c[j] { goto Loop }  
        }  
    }
```

Problema da Seção Crítica

Soluções em Hardware, ~1980s

```
atomic func TestAndSet(ref *bool, bit bool) (old bit) {  
    old = *ref  
    *ref = bit  
    return old  
}
```

Mutexes

```
type Mutex  
func Lock(&Mutex)  
func Unlock(&Mutex)
```

```
typedef struct { ... } mtx_t;  
int mtx_init(mtx_t*, int);  
int mtx_lock(mtx_t*);  
int mtx_unlock(mtx_t*);  
int mtx_destroy(mtx_t*);
```

```
class ReentrantLock {  
    public ReentrantLock();  
    public void lock();  
    public void unlock();  
}
```

```
class Lock:  
    def __init__(self): ...  
    def __enter__(self) -> None: ...  
    def __exit__(self) -> None: ...
```

```
lock = Lock()  
with lock: ...
```

Variáveis Condicionais

```
type Cond { &Mutex }  
func Wait(&Cond)  
func Signal(&Cond)  
func Broadcast(&Cond)
```

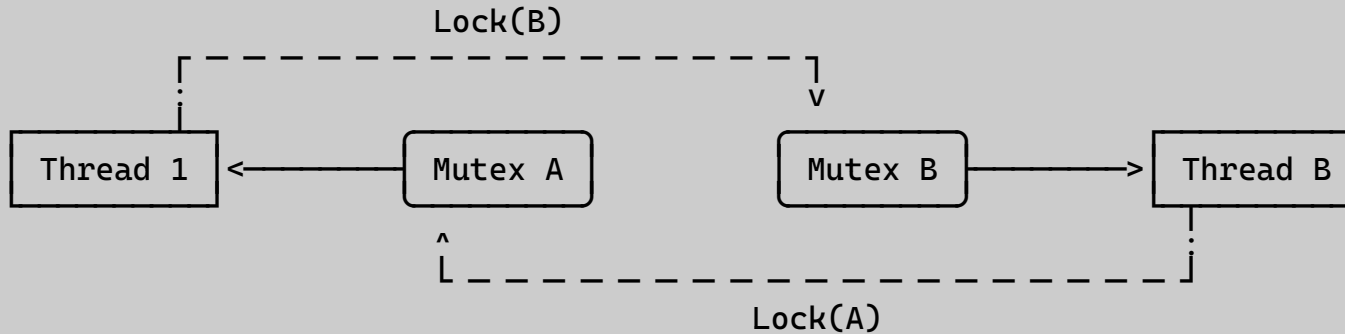
```
typedef struct { ... } cnd_t;  
int cnd_init(cnd_t*);  
int cnd_wait(cnd_t*, mtx_t*);  
int cnd_signal(cnd_t*);  
int cnd_broadcast(cnd_t*);  
int cnd_destroy(cnd_t*);
```

```
class ReentrantLock {  
    public Condition newCondition();  
}  
  
class Condition {  
    public void await();  
    public void signal();  
    public void signalAll();  
}
```

```
class Condition(Lock):  
    def __init__(self, lock: Lock|None = None): ...  
    def __enter__(self) -> None: ...  
    def __exit__(self) -> None: ...  
    def wait(self) -> None: ...  
    def notify(self) -> None: ...  
    def notify_all(self) -> None: ...
```

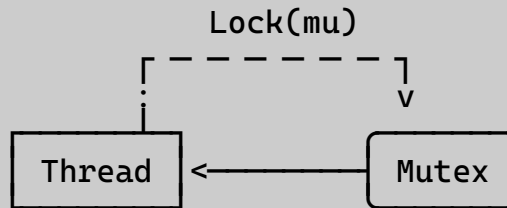
Deadlocks

Deadlock é uma classe de bug de software, este acontece quando threads esperam mutuamente sobre recursos que outras threads seguram.



Deadlocks

Deadlock é uma classe de bug de software, este acontece quando threads esperam mutuamente sobre recursos que outras threads seguram.



Notas Bibliográficas

- DIJKSTRA, Edgar. Solution of a Problem in Concurrent Programming Control. **Communications of the ACM**. 1965. <hyperlink>.
- BIRRELL, Andrew. An Introduction to Programming with Threads. **Digital Equipment Corporation**. 1989. <hyperlink>.

It is now safe to power off your computer.